

Linux Kernel Module for Listing Tasks

- In this project, you will write a kernel module that **lists all current tasks** in a Linux system.

Part I—Iterating over Tasks Linearly

- Design a kernel module that iterates through all tasks in the system using the **for_each_process()** macro(in <linux/sched/signal.h>).
- In particular, output the **process id**, **task name** (known as *executable name*), and **state** of each task.
- You will probably have to read through the **task_struct** structure in <linux/sched.h> to obtain the names of these fields.
- Write this code in the module entry point so that its contents will appear in the **kernel log buffer**, which can be viewed using the **dmesg** command.

```
os@os-VirtualBox: ~/桌面/HW4/Linear
[ 5749.665199] pid: 1 | pname: systemd | state: 1
[ 5749.665200] pid: 2 | pname: kthreadd | state: 1
[ 5749.665200] pid: 4 | pname: kworker/0:0H | state: 1
[ 5749.665201] pid: 6 | pname: mm_percpu_wq | state: 1
[ 5749.665202] pid: 7 | pname: ksoftirqd/0 | state: 1
[ 5749.665203] pid: 8 | pname: rcu_sched | state: 1
[ 5749.665204] pid: 9 | pname: rcu_bh | state: 1
[ 5749.665205] pid: 10 | pname: migration/0 | state: 1
[ 5749.665206] pid: 11 | pname: watchdog/0 | state: 1
[ 5749.665207] pid: 12 | pname: cpuhp/0 | state: 1
[ 5749.665208] pid: 13 | pname: kdevtmpfs | state: 1
[ 5749.665209] pid: 14 | pname: netns | state: 1
[ 5749.665209] pid: 15 | pname: khungtaskd | state: 1
[ 5749.665210] pid: 16 | pname: oom_reaper | state: 1
[ 5749.665211] pid: 17 | pname: writeback | state: 1
[ 5749.665212] pid: 18 | pname: kcompactd0 | state: 1
[ 5749.665213] pid: 19 | pname: ksm | state: 1
[ 5749.665214] pid: 20 | pname: khugepaged | state: 1
[ 5749.665215] pid: 21 | pname: crypto | state: 1
[ 5749.665216] pid: 22 | pname: kintegrityd | state: 1
[ 5749.665216] pid: 23 | pname: kblockd | state: 1
[ 5749.665217] pid: 24 | pname: ata_sff | state: 1
[ 5749.665218] pid: 25 | pname: md | state: 1
[ 5749.665219] pid: 26 | pname: edac-poller | state: 1
[ 5749.665220] pid: 27 | pname: devfreq_wq | state: 1
[ 5749.665221] pid: 28 | pname: watchdogd | state: 1
[ 5749.665222] pid: 32 | pname: kauditd | state: 1
[ 5749.665223] pid: 33 | pname: kswapd0 | state: 1
[ 5749.665224] pid: 34 | pname: ecryptfs-kthrea | state: 1
[ 5749.665224] pid: 76 | pname: kthrotld | state: 1
[ 5749.665225] pid: 77 | pname: acpi_thermal_pm | state: 1
[ 5749.665226] pid: 78 | pname: scsi_eh_0 | state: 1
[ 5749.665227] pid: 79 | pname: scsi_tmf_0 | state: 1
[ 5749.665228] pid: 80 | pname: scsi_eh_1 | state: 1
[ 5749.665229] pid: 81 | pname: scsi_tmf_1 | state: 1
[ 5749.665229] pid: 87 | pname: ipv6_addrconf | state: 1
[ 5749.665230] pid: 112 | pname: charger_manager | state: 1
[ 5749.665231] pid: 163 | pname: kworker/0:1H | state: 1
[ 5749.665232] pid: 164 | pname: scsi_eh_2 | state: 1
[ 5749.665233] pid: 165 | pname: scsi_tmf_2 | state: 1
[ 5749.665234] pid: 190 | pname: jbd2/sda1-8 | state: 1
[ 5749.665235] pid: 191 | pname: ext4-rsv-conver | state: 1
[ 5749.665235] pid: 220 | pname: systemd-journal | state: 1
[ 5749.665236] pid: 243 | pname: systemd-udev | state: 1
[ 5749.665237] pid: 294 | pname: iprt-VBoxWQueue | state: 1
[ 5749.665238] pid: 295 | pname: systemd-timesyn | state: 1
[ 5749.665239] pid: 341 | pname: ttm_swap | state: 1
[ 5749.665239] pid: 632 | pname: ModemManager | state: 1
[ 5749.665240] pid: 634 | pname: avahi-daemon | state: 1
[ 5749.665241] pid: 635 | pname: dbus-daemon | state: 1
[ 5749.665242] pid: 644 | pname: avahi-daemon | state: 1
[ 5749.665243] pid: 650 | pname: NetworkManager | state: 1
[ 5749.665244] pid: 669 | pname: snapd | state: 1
```

- To verify that your code is working correctly, compare the contents of the kernel log buffer with the output of the following command, which lists all tasks in the system:

ps -el

- The two values should be very similar. Because tasks are dynamic, however, it is possible that a few tasks may appear in one listing but not the other.

Part II—Iterating over Tasks with a Depth-First Search Tree

- The second portion of this project involves iterating over all tasks in the system using a **depth-first search** (DFS) tree.

- Examining the task struct in <linux/sched.h>, we see two struct list head objects: **children** and **sibling**.
- These objects are pointers to a list of the task's children, as well as its siblings.

- Beginning from the `init_task`, design a kernel module that iterates over all tasks in the system using a DFS tree.
- Just as in the first part of this project, output the pid, name, and state of each task.
- Perform this iteration in the kernel entry module so that its output appears in the kernel log buffer.

```
os@os-VirtualBox: ~/桌面/HW4/DFS
[ 6110.594296] pid: 1 | pname: systemd | state: 1
[ 6110.594297] pid: 220 | pname: systemd-journal | state: 1
[ 6110.594297] pid: 243 | pname: systemd-udev | state: 1
[ 6110.594298] pid: 295 | pname: systemd-timesyn | state: 1
[ 6110.594299] pid: 632 | pname: ModemManager | state: 1
[ 6110.594300] pid: 634 | pname: avahi-daemon | state: 1
[ 6110.594301] pid: 644 | pname: avahi-daemon | state: 1
[ 6110.594301] pid: 635 | pname: dbus-daemon | state: 1
[ 6110.594302] pid: 650 | pname: NetworkManager | state: 1
[ 6110.594303] pid: 823 | pname: dhclient | state: 1
[ 6110.594304] pid: 836 | pname: dnsmasq | state: 1
[ 6110.594305] pid: 669 | pname: snapd | state: 1
[ 6110.594306] pid: 671 | pname: accounts-daemon | state: 1
[ 6110.594306] pid: 672 | pname: rsyslogd | state: 1
[ 6110.594307] pid: 675 | pname: systemd-logind | state: 1
[ 6110.594308] pid: 695 | pname: cron | state: 1
[ 6110.594309] pid: 701 | pname: acpid | state: 1
[ 6110.594310] pid: 770 | pname: polkitd | state: 1
[ 6110.594310] pid: 781 | pname: lightdm | state: 1
[ 6110.594311] pid: 821 | pname: Xorg | state: 1
[ 6110.594312] pid: 988 | pname: lightdm | state: 1
[ 6110.594313] pid: 997 | pname: upstart | state: 1
[ 6110.594313] pid: 1081 | pname: upstart-udev-br | state: 1
[ 6110.594314] pid: 1083 | pname: dbus-daemon | state: 1
[ 6110.594315] pid: 1095 | pname: window-stack-br | state: 1
[ 6110.594315] pid: 1112 | pname: upstart-dbus-br | state: 1
[ 6110.594316] pid: 1114 | pname: upstart-dbus-br | state: 1
[ 6110.594316] pid: 1117 | pname: upstart-file-br | state: 1
[ 6110.594317] pid: 1132 | pname: fcitx | state: 1
[ 6110.594318] pid: 1138 | pname: dbus-daemon | state: 1
[ 6110.594319] pid: 1149 | pname: fcitx-dbus-watc | state: 1
[ 6110.594319] pid: 1151 | pname: bamfd daemon | state: 1
[ 6110.594320] pid: 1142 | pname: gnome-keyring-d | state: 1
[ 6110.594321] pid: 1205 | pname: gvfsd | state: 1
[ 6110.594322] pid: 1212 | pname: gvfsd-fuse | state: 1
[ 6110.594323] pid: 1218 | pname: gpg-agent | state: 1
[ 6110.594323] pid: 1216 | pname: at-spi-bus-laun | state: 1
[ 6110.594324] pid: 1223 | pname: dbus-daemon | state: 1
[ 6110.594325] pid: 1226 | pname: at-spi2-registr | state: 1
[ 6110.594326] pid: 1186 | pname: notify-osd | state: 1
[ 6110.594327] pid: 1268 | pname: hud-service | state: 1
[ 6110.594327] pid: 1270 | pname: unity-settings- | state: 1
[ 6110.594328] pid: 1279 | pname: gnome-session-b | state: 1
[ 6110.594329] pid: 1640 | pname: nautilus | state: 1
[ 6110.594330] pid: 1649 | pname: polkit-gnome-au | state: 1
[ 6110.594330] pid: 1661 | pname: gnome-software | state: 1
[ 6110.594331] pid: 1667 | pname: nm-applet | state: 1
[ 6110.594332] pid: 1668 | pname: unity-fallback- | state: 1
[ 6110.594333] pid: 1893 | pname: zeitgeist-datah | state: 1
[ 6110.594334] pid: 2039 | pname: update-notifier | state: 1
[ 6110.594334] pid: 2359 | pname: deja-dup-monito | state: 1
[ 6110.594335] pid: 1283 | pname: unity-panel-ser | state: 1
[ 6110.594336] pid: 1344 | pname: dconf-service | state: 1
[ 6110.594337] pid: 1368 | pname: indicator-messa | state: 1
```

- To check the output of the DFS tree, use the command
ps -eLf

- hw4_{studentID}.rar:
 - hw4_linear.c (45%)
 - hw4_dfs.c (45%)
 - hw3_report(10%)
 - Tell us how you implement your homework and show us your results.
- 0 will be given to cheaters, so don't copy & paste your friend's code directly.
- Deadline:6/19(Tue.) 23:59
- Delay is allowed.
 - $\text{score} = (1 - 10\% * \text{\#delay days}) * \text{original score}.$